

No-Load Solar Panel Testing And Monitoring System

Ryheeme Donegan
University of The West Indies
Department of Physics
Electronics Research Project Report
Dr. Jayaka Campbell
ryheeme.donegan@mymona.uwi.edu

Abstract

This project presents SolarSense, a portable system for testing and monitoring key parameters of solar panels under no-load conditions. The device is rated at 24V and 30A and measures a panel's open-circuit voltage (Voc), short-circuit current (Isc), panel temperature, and solar irradiance, providing quick diagnostic data without the need to connect the panel to a load. The system integrates voltage and current sensing circuits, two temperature sensors, and a light dependent resistor (LDR) with a WiFi enabled microcontroller for real-time data acquisition and remote displaying. Measurements are also displayed on an onboard web interface and is logged for future analysis. This device is intended for use in solar panel maintenance, quality control, and research, offering a convenient method to evaluate panels in the field and identify potential issues before deployment or during service checks.

Contents

1. Introduction	3
2. Methodology	3
2.1. System Overview	3
2.2. System Architecture	3
2.2.1. Block Diagram	3
2.2.2. Flow Chart	4
2.2.3. Schematic Diagram	4
2.3. Communication And Wireless Protocols	4
2.3.1. WiFi	4
2.3.2. MQTT	4
2.3.3. NTP	5
2.4. Hardware Components	5
2.4.1. Microcontroller	5
2.4.2. Sensors	5
2.4.2.1. Current	5
2.4.2.2. Temperature	5
2.4.2.3. Irradiance	6
2.4.2.4. Voltage	6
2.5. Firmware And Software Components	6
2.5.1. Hardware Code	6
2.5.2. Webpages	6
2.5.3. Database	7
2.6. System Operation and Data Acquisition	7
2.6.1. Startup And Initialization	7
2.7. Testing And Validation	7
2.7.1.	7
3. Results	7
4. Discussion	7
5. Conclusion	7
Bibliography	7
6. Appendix	7
6.1. Firmware For ESP32	7

1. Introduction

Solar energy has become a pivotal technology in sustainable power generation, with solar panels being the primary energy conversion component. The efficiency and reliability of these panels are necessary for the performance of any solar power system. However, accurately assessing a panel's health and performance in the field can be challenging, often requiring complex and bulky equipment.

Conventional techniques usually entail attaching panels to loads, which is a laborious procedure that isn't always feasible for on-site inspections or quick diagnostics. This project presents SolarSense, an innovative, portable device made to simplify solar panel diagnostics.

SolarSense offers a thorough, load-free solar panel testing solution that significantly increases efficiency and convenience. Open-circuit voltage (Voc), short-circuit current (Isc), panel temperature, and sun irradiance are among the important electrical and environmental characteristics that may be measured by this specialized instrument, which has a 24V and 30A rating. A WiFi-enabled microcontroller serves as the central component of the system, integrating two temperature sensors, a light-dependent resistor (LDR) to collect data in real time, and bespoke circuits for voltage and current sensing. Every measurement is recorded for future study and shown on an easy-to-use onboard web interface. SolarSense is an invaluable instrument for solar panel maintenance, quality control, and research because it provides instant diagnostic data without requiring a connected load. This allows technicians and researchers to assess panel performance and spot possible problems with unmatched ease and portability.

2. Methodology

2.1. System Overview

SolarSense is portable, light-weight and self-contained device designed to perform diagnostic testing and monitoring of solar panels under no-load conditions. The system's architecture is built around a WiFi-enabled microcontroller (ESP32-S3) that integrates three primary subsystems: data acquisition, data processing, and a user interface. The data acquisition subsystem consists of various sensors and circuits that measure the panel's open-circuit voltage, short-circuit current, panel temperature, and solar irradiance. The microcontroller samples and process this data and presents it to the user in real-time through an onboard and remote web interface. Data is also logged on an external database which allows for historical analysis to be performed.

2.2. System Architecture

2.2.1. Block Diagram

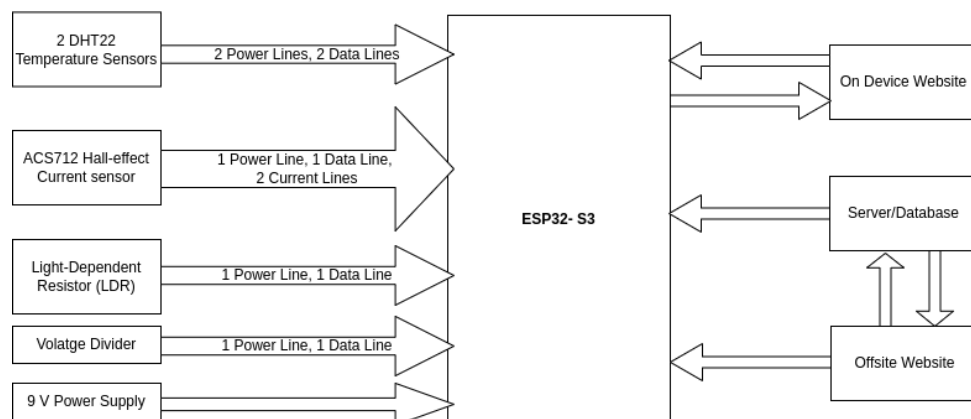


Figure 1: Block Diagram of System.

This block diagram illustrates the connections and data flow of the solar panel monitoring system. An ESP32-S3 microcontroller powered by a 9V supply is at the center, serving as the main processing unit. It receives input from several sensors such as two DHT22 temperature sensors, an ACS712 Hall-effect current sensor and a Light-Dependent Resistor (LDR).

The ESP32-S3 processes this sensor data and then outputs it to three destinations: an on device website, a remote server and database, and a separate offsite website. The diagram also shows the specific number of power and data lines required for each sensor.

2.2.2. Flow Chart

2.2.3. Schematic Diagram

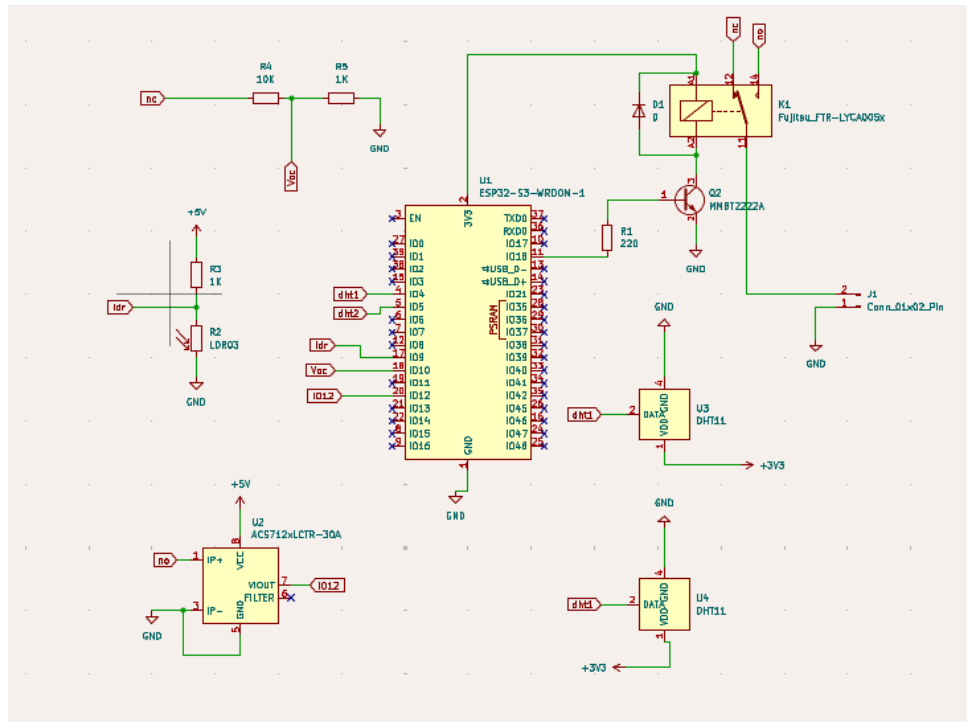


Figure 2: Schematic Diagram of System.

2.3. Communication And Wireless Protocols

2.3.1. WiFi

2.3.2. MQTT

In order to facilitate reliable communication between the system and a remote server, the Message Queuing Telemetry Transport (MQTT) protocol was chosen. This protocol was selected for its native support on the ESP32 and its robust reliability, which is provided by Quality of Service (QoS) levels to ensure message delivery over unstable networks. MQTT also allows messages to be queued and resent if a loss of network connection is detected. The WebSocket protocol was considered as an alternative; however, after a comparison of both protocols, as shown in the table below, demonstrates MQTT's suitability for this project's IoT focus. [1].

Feature	MQTT	WebSocket
Architecture	Publish/Subscribe Model	Bidirectional, Socket-like API
Communication Type	Asynchronous (one-to-many)	Asynchronous (one-to-one)
Connection Type	Long-lived connections via broker	Persistent direct connections

Message Queuing	Supported by the broker	No native queuing
Primary Use Case	IoT, M2M Communication	Real-time Web Applications

2.3.3. NTP

The Network Time Protocol (NTP) is a sub-millisecond time synchronization protocol designed for constrained devices like the ESP32 microcontroller [2]. Due to its accuracy and lightweight nature, NTP has a high prevalence in IoT projects and was therefore chosen for the system. The protocol allows the microcontroller to synchronize its internal clock with a global time server, allowing the system to put accurate timestamps to any data acquired by the sensors. These accurate timestamps are crucial for performing historical analysis and correlating sensor data with specific times of day or across any designated time period.

2.4. Hardware Components

2.4.1. Microcontroller

The ESP32-S3 microcontroller (mcu) was chosen to power the system. This mcu provides integrated WiFi, up to 8MB of storage and the FreeRTOS operating system allowing for efficient and fast multi-tasking [3].

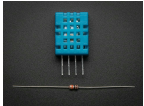
2.4.2. Sensors

2.4.2.1. Current

The short-circuit current was measured using the ACS712 Hall-effect current sensor. Even tho a shunt resistor with a differential amplifier is a common alternative for measuring current, the ACS712 was chosen for its key advantage of galvanic isolation, which enhances the safety of the system by separating the high current output from the microcontroller. The ACS712ELCTR-30A variant was used because of its high input current and simple design. [4]

2.4.2.2. Temperature

Two DHT22 temperature sensors were used to measure the panel's operating temperature and ambient temperature. The DHT22 was chosen for its cost-effectiveness and its superior accuracy when compared to similar low-cost sensors like the DHT11 [5]. A key factor in the sensor choice was the fact that the DHT22 communicates over 1-wire and not analog outputs. This digital approach was chosen because it offers superior noise immunity over long wires [6]. This fact made the DHT22 favorable over sensors such as the LM35 and the LM335. The DHT22 has a temperature accuracy of ± 0.5 degrees C, providing the necessary precision for the project's requirements [7]. See table 1, for a detailed comparison of the two sensors.

Sensor		Accuracy	Parameters
DHT11		$\pm 2^{\circ}\text{C}$	Temperature Range: $0 - 50^{\circ}\text{C}$ Voltage: $3 - 5\text{V}$
DHT22		$\pm 0.5^{\circ}\text{C}$	Temperature Range: $-40 - 80^{\circ}\text{C}$ Voltage: $3 - 5\text{V}$

2.4.2.3. Irradiance

The irradiance of a solar panel can be accurately measured using a light-dependent resistor (LDR) in a voltage divider circuit, a method outlined by Barros et al. [8].

The procedure involved placing a 1kΩ resistor in series with the LDR. For every voltage reading V_1 measured by the ADC, the following equation was used to calculate the corresponding irradiance

$$G_i = 47.5V_1 - 4.4 \times 102V_i + 1048$$

where G_i is the irradiance in $\frac{W}{m^2}$

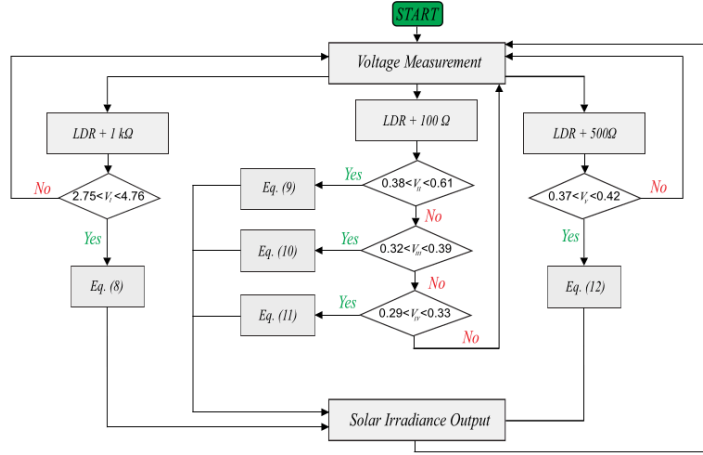


Figure 3: Flowchart showing the steps to measure irradiance using the chosen resistor and LDR.

Adapted from Barros et al. [8]

2.4.2.4. Voltage

In order to measure the open-circuit voltage, a voltage divider circuit was constructed. This circuit consisted of a 10kΩ resistor (R_1) and a 1kΩ resistor (R_2) in series. The purpose of this circuit was to safely scale down the 24V supply to an output voltage of approximately 2.18V, which is well within the 3.3V range of the ESP32's ADC. The voltage divider formula is shown below. The divider formula is shown below.

$$V_o = \frac{R_1}{R_1 + R_2} V_s$$

Where V_o is the output voltage, V_s is the source voltage, R_1 is the resistor connected to the ground and R_2 is the resistor connected to the source voltage.

2.5. Firmware And Software Components

In order to minimize development time and software size and to maximize system efficiency and usability, the follow software decisions were made.

2.5.1. Hardware Code

2.5.2. Webpages

Alpine.js, PicoCSS and standard HTML were chosen for the both the onboard and offsite websites of the system. These technologies provided a balance among development speed, size and efficiency. Alpine.js was chosen over technologies such as React or Vue because of its miniature size and the ability to be used without a build step. This lack of a build step was instrumental in embedding a webpage directly on the ESP32.

The Golang programming language was chosen because of its built-in server libraries and the ability to natively embed files into a single binary capable of running on all major operating systems such as Linux, MacOS and Windows.

2.5.3. Database

SQLite was chosen over other

2.6. System Operation and Data Acquisition

2.6.1. Startup And Initialization

Upon boot, the microcontroller initializes the wireless connections used throughout the operation. These connections include connecting to a WiFi network in order to initiate communication with an external MQTT broker.

2.7. Testing And Validation

2.7.1.

Talk about the esp32 and other components and why those were chosen and what alternatives, and have the block diagram and flow chart for the system

3. Results

4. Discussion

5. Conclusion

Bibliography

- [1] EMQX, "MQTT vs WebSocket: Key Differences & Applications." 2024.
- [2] B. K. D. R, B. B. Sinha, and G. R, "Clock synchronization in industrial Internet of Things and potential works in precision time protocol: Review, challenges and future directions," *International Journal of Cognitive Computing in Engineering*, vol. 4, pp. 205–219, 2023, doi: <https://doi.org/10.1016/j.ijcce.2023.06.001>.
- [3] Espressif Systems, "ESP32-S3 Series Datasheet Version 2.0." 2016.
- [4] Allegro MicroSystems, "ACS712 Fully Integrated, Hall Effect-Based Linear Current Sensor." 2019.
- [5] Adafruit Industries, "DHT11 Temperature-Humidity Sensor." [Online]. Available: <https://www.adafruit.com/product/386>
- [6] B. Linke, "Overview of 1-Wire Technology and Its Use." 2008.
- [7] Adafruit Industries, "DHT22 Temperature-Humidity Sensor." 2024.
- [8] D. do Carmo Mendonça, R. de Barros, W. Amorim, H. Pereira, and J. M. Callegari, "Low-Cost Solar Irradiance Meter using LDR Sensors," 2018, p. . doi: 10.1109/INDUSCON.2018.8627176.

6. Appendix

6.1. Firmware For ESP32

```
#include <Arduino.h>
#include <WiFi.h>
#include "time.h"
#include <DHT_U.h>
```

```

#include <DHT.h>
#include <PubSubClient.h>
#include <Adafruit_Sensor.h>
#include <ESPAsyncWebServer.h>
#include <ArduinoJson.h>

#define SENSOR_1_TYPE    DHT22
#define SENSOR_2_TYPE    DHT22

const char* ntp = "pool.ntp.org";
const char* ssid = "Device";
const char* password = "password0";

const char *broker = "10.67.53.70";
const int port = 1883;

const long  gmtoffset_sec = -5 * 3600;
const int  daylightOffset_sec = 0;

const float voltage_offset = 11.83;
const float voltage_error = 2.105;
const int  current_offset = 2360;

typedef struct tm time_struct;
typedef time_t time_stamp;
typedef int adc_input;
typedef int adc_value;
typedef int digital_pin;

sensor_t sensor_1;
sensor_t sensor_2;

time_stamp getCurrentUnixTime();

void readADCPins();
void publishData();
float getIrradiance(float voltage);
String getWebsite();

time_stamp currentUnixTime = 0;

const adc_input ldr_pin = 9;
const adc_input voltage_pin = 10;
const adc_input current_pin = 12;
const digital_pin transistor_base_pin = 18;
const int sensor1_pin = 4;
const int sensor2_pin = 5;
const uint16_t TFT_CS = 26;
const uint16_t TFT_RST = 47;

DHT_Unified dht(sensor1_pin, SENSOR_1_TYPE);
DHT_Unified dht2(sensor2_pin, SENSOR_2_TYPE);

```



```

sensors_event_t event_1;
sensors_event_t event_2;

WiFiClient espClient;
AsyncWebServer server(80);
PubSubClient client(espClient);

// PubSubClient client(espClient);

float ldr_value = 0;
adc_value voltage_value = 0;
float current_value = 0.0;

float temperature_value = 0;
float temperature2_value = 0;

void enablePins(){
    pinMode(transistor_base_pin,OUTPUT);
    digitalWrite(transistor_base_pin,LOW);
}

void setup() {

    Serial.begin(9600);
    Serial.println("Hello");
    dht.begin();
    dht2.begin();

    enablePins();

    WiFi.begin(ssid,password);
    while (!WiFi.isConnected()) {
        Serial.println("Connecting to wifi");
        delay(200);
    }

    Serial.println("Connected to wifi");
    Serial.print("IP Address: ");
    Serial.println(WiFi.localIP());

    client.setServer(broker,port);
    int retryCount = 0;
    while(!client.connect("test1")){
        Serial.print("MQTT connection attempt ");
        Serial.print(retryCount + 1);
        Serial.print(" failed. State: ");
        Serial.println(client.state());
        delay(1000);
        retryCount++;
    }

    server.on("/", HTTP_GET, [](AsyncWebServerRequest *request){

```

```

    request->send(200, "text/html", getWebsite());
});

server.on("/data", HTTP_GET, [](AsyncWebServerRequest *request){
    JsonDocument doc;

    doc["temperature"] = (temperature_value + temperature2_value) / 2;
    doc["voc"] = voltage_value;
    doc["isc"] = current_value;
    doc["irr"] = getIrradiance(ldr_value);
    doc["irr"] = ldr_value;
    doc["timestamp"] = currentUnixTime;

    String jsonString;
    serializeJson(doc, jsonString);

    request->send(200, "application/json", jsonString);
});

server.begin();

configTime(gmtOffset_sec, daylightOffset_sec, ntp);

delay(1000);
}

void loop() {

    if (Serial.available() > 0) {
        char incomingByte = Serial.read();

        if (incomingByte == '1') {
            Serial.println("Received command '1'");
            digitalWrite(transistor_base_pin, !digitalRead(transistor_base_pin));
        }
    }

    dht.temperature().getEvent(&event_1);
    if (isnan(event_1.temperature)) {
        Serial.println(F("Error reading temperature!"));
    }
    else {
        // Serial.print(F("Temperature: "));
        // Serial.print(event_1.temperature);
        temperature_value = event_1.temperature;
        // Serial.println(F("°C"));
    }

    delay(200);

    dht2.temperature().getEvent(&event_2);
    if (isnan(event_2.temperature)) {

```

```

        Serial.println(F("Error reading temperature!"));
    }
    else {
        // Serial.print(F("Temperature: "));
        // Serial.print(event_2.temperature);
        temperature2_value = event_2.temperature;
        // Serial.println(F("°C"));
    }

    currentUnixTime= getCurrentUnixTime();
    // Serial.println(currentUnixTime);

    readADCPins();

    // Serial.print("Voc: ");
    // Serial.print(voltage_value);
    // Serial.println("mV");

    // Serial.print("Isc: ");
    // Serial.print(current_value);
    // Serial.println("A");

    publishData();

    delay(1000);

    client.loop();

}

void publishData(){
    // Check if MQTT is connected
    if (!client.connected()) {
        // Serial.println("MQTT not connected, cannot publish");
        // Serial.print("Client state: ");
        // Serial.println(client.state());
        return;
    }

    JsonDocument doc;
    doc["temperature"] = (temperature_value + temperature2_value) / 2;
    doc["voc"] = voltage_value;
    doc["isc"] = current_value;
    doc["irr"] = ldr_value;
    doc["timestamp"] = currentUnixTime;
    doc["ip"] = WiFi.localIP();

    String jsonString;
    serializeJson(doc, jsonString);

    // Serial.println("Publishing data to MQTT broker...");
    // Serial.print("Topic: rylimitless/dev, Payload: ");

```

```

// Serial.println(jsonString);
// Serial.print("Payload length: ");
// Serial.println(jsonString.length());

bool published = client.publish("rylimitless/dev", jsonString.c_str());
if(published) {
    Serial.println("Published successfully");
} else {
    Serial.println("Failed to publish");
    // Serial.print("Client state after publish attempt: ");
    // Serial.println(client.state());
}
}

void readADCPins(){

    adc_input val = 0;
    for(int i=0;i < 40;i++){
        val += analogRead(ldr_pin);
        delay(10);
    }

    voltage_value = analogRead(voltage_pin) * 3280 * voltage_offset / 4095;

    digitalWrite(transistor_base_pin, !digitalRead(transistor_base_pin));

    delay(700);

    adc_input val2 = 0;
    for(int i=0;i<100;i++){
        val2+= analogRead(current_pin);
        delay(10);
    }

    digitalWrite(transistor_base_pin, !digitalRead(transistor_base_pin));

    delay(700);

    val2 = val2 / 100;

    // current_value = ((val2 * 3280 / 4095) - current_offset) / 66.0;
    current_value = ((val2 * 3280 / 4095) - current_offset) / 66.0;
    if(current_value < 0){

        current_value = 0;
    }

    adc_input val3 = 0;
    for(int i=0;i<40;i++){
        val3+= analogRead(ldr_pin);
        delay(10);
    }
}

```

```

ldr_value = ((val3 / 40) * 3.28 / 4095) * 7.7;
Serial.println(ldr_value);

ldr_value = getIrradiance(ldr_value);
Serial.println(ldr_value);
}

float getIrradiance(float voltage) {

    if (voltage > 2.75 && voltage < 4.76){
        return (47.5 * voltage * voltage) - (4.4 * 100 * voltage) + 1048;
    }
    return -999;
}

time_stamp getCurrentUnixTime(){
    time_stamp currentTime;
    time(&currentTime);
    if(currentTime < 10) return -1;
    return currentTime;
}

String getWebsite() {

    return R"HTML(<!doctype html>
<html lang="en" data-theme="dark">
    <head>
        <meta charset="utf-8" />
        <meta name="viewport" content="width=device-width, initial-scale=1" />
        <meta name="color-scheme" content="light dark" />
        <link
            rel="stylesheet"
            href="https://cdn.jsdelivr.net/npm/@picocss/pico@2/css/pico.min.css"
        />
    <style>
        .display-card {
            --pico-card-background-color: #1a1a1a;
            border-left: 4px solid var(--pico-primary);
            transition: transform 0.2s ease-in-out;
        }
        .display-card:hover {
            transform: translateY(-5px);
        }
        .value {
            font-size: 2.5rem;
            font-weight: bold;
            color: var(--pico-primary);
            margin: 0;
            line-height: 1;
        }
        .unit {

```

```

        font-size: 1rem;
        color: var(--pico-contrast-500);
    }
    .title {
        display: flex;
        align-items: center;
        gap: 0.5rem;
        font-size: 1.2rem;
        font-weight: 500;
        color: var(--pico-contrast-800);
    }
    .title .emoji {
        font-size: 1.5rem;
    }
</style>
<script
    defer
    src="https://cdn.jsdelivr.net/npm/alpinejs@3.x.x/dist/cdn.min.js"
></script>
<script>
    document.addEventListener("alpine:init", () => {
        Alpine.data("data", () => ({
            data: {
                temperature: -999,
                voc: -999,
                isc: -999,
                irr: -999,
            },
            intervalId: null,

            init() {
                console.log(
                    "Alpine.js component initialized. Fetching data...",
                );
                this.startFetchingData();
            },

            async getData() {
                try {
                    const response = await fetch("http://10.67.53.182:80/
data");

                    if (!response.ok) {
                        throw new Error(
                            `HTTP error! Status: ${response.status}`,
                        );
                    }

                    const newData = await response.json();

                    console.log(newData);

                    this.data.temperature = newData.temperature;
                    this.data.voc = newData.voc / 1000;
                    this.data.isc = newData.isc;
                    this.data.irr = newData.irr;

```

```

        console.log("Data updated successfully:", newData);
    } catch (error) {
        console.error("Error fetching data:", error);

        this.data.temperature = -999;
        this.data.voc = -999;
        this.data.isc = -999;
        this.data.irr = -999;
    }
},

startFetchingData() {
    this.getData();
    this.intervalId = setInterval(() => {
        this.getData();
    }, 2000);
},

destroy() {
    clearInterval(this.intervalId);
},

}));
});
</script>
<title>SolarSence On-Device Monitoring</title>
</head>
<body>
    <main class="container">
        <h1>SolarSence On-Device Monitoring</h1>
        <br>
        <h3> Please Note, A Value of -999 Signifies and error'd reading. </h3>
        <div x-data="data">
            <div class="grid">
                <article class="display-card">
                    <header>
                        <hgroup>
                            <p class="title">
                                <span class="emoji">🌡️</span>Temperature
                            </p>
                        </hgroup>
                    </header>
                    <footer>
                        <p class="value">
                            <span
                                x-text="data.temperature.toFixed(1)"
                            ></span>
                            <span class="unit">°C</span>
                        </p>
                    </footer>
                </article>

                <article class="display-card">
                    <header>
                        <hgroup>
                            <p class="title">

```

```

        <span class="emoji"></span>Open-Circuit
        Voltage
    </p>
</hgroup>
</header>
<footer>
    <p class="value">
        <span x-text="data.voc.toFixed(2)"></span>
        <span class="unit">V</span>
    </p>
</footer>
</article>

<article class="display-card">
    <header>
        <hgroup>
            <p class="title">
                <span class="emoji">💡</span>Short-Circuit
                Current
            </p>
        </hgroup>
    </header>
    <footer>
        <p class="value">
            <span x-text="data.isc.toFixed(3)"></span>
            <span class="unit">A</span>
        </p>
    </footer>
</article>

<article class="display-card">
    <header>
        <hgroup>
            <p class="title">
                <span class="emoji">✖️</span>Irradiance
            </p>
        </hgroup>
    </header>
    <footer>
        <p class="value">
            <span x-text="data.irr.toFixed(2)"></span>
            <span class="unit">W/m²</span>
        </p>
    </footer>
</article>
</div>
</div>
</main>
</body>
</html>

```

```

)HTML";

```

```

}

```